

Lecture 17: Signed Number Representation (Unsigned, Sign-Magnitude, 1's Complement & 2's Complement)

Learning Objectives

By the end of this lecture, students will be able to:

- Understand why signed number representation is needed.
 - Differentiate between Unsigned and Signed numbers.
 - Represent numbers using Sign-Magnitude.
 - Find the 1's Complement of a binary number.
 - Find the 2's Complement of a binary number.
 - Represent negative numbers using 2's Complement.
 - Determine the range of signed and unsigned numbers.
 - Understand why modern computers use 2's Complement.
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally mention:

"In today's Computer Organization and Assembly Language lecture, we'll learn Signed Number Representation, including Unsigned Numbers, Sign-Magnitude, 1's Complement, and 2's Complement, with

easy examples to understand how computers store positive and negative numbers."

In the previous lecture, we learned how to perform Binary Addition, Subtraction, Multiplication, and Division.

However, those examples only involved positive numbers.

But what happens when we need to represent negative numbers like -5 or -20?

Computers only understand Binary digits (0 and 1). They don't have a special "-" symbol like humans do.

So, how does a computer know whether a number is positive or negative?

The answer lies in **Signed Number Representation**.

In today's lecture, we'll learn four methods of representing numbers: Unsigned Representation, Sign-Magnitude, 1's Complement, and 2's Complement. We'll also see why almost all modern processors use 2's Complement.

Let's begin.

2. Why Do We Need Signed Numbers?

Many real-world applications use negative values.

Examples:

- Bank balance: **-500**
- Temperature: **-8°C**
- Elevation: **-50 m**
- Profit/Loss calculations
- Scientific computations

A computer must be able to store both **positive** and **negative** numbers.

3. Unsigned Numbers

Definition

Unsigned numbers can represent only zero and positive values.

Example using **4 bits**:

Binary	Decimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8

1111	15
------	----

Formula

For **n bits**:

Range = 0 to $(2^n - 1)$

Example:

8 bits

$0 \rightarrow 255$

4. Signed Numbers

A signed number can represent:

- Positive numbers
- Negative numbers
- Zero

The **Most Significant Bit (MSB)** is commonly used as the **Sign Bit**.

0 = Positive

1 = Negative

Example (8-bit):

00000101 = +5

10000101 = Negative number

But how do we determine its value?

That depends on the representation method.

5. Sign-Magnitude Representation

Rule

- MSB = Sign
- Remaining bits = Magnitude

0 → Positive

1 → Negative

Example:

+5

00000101

-5

10000101

Problem with Sign-Magnitude

There are **two representations of zero**.

+0

00000000

-0

10000000

Having two different representations for zero is inefficient and complicates arithmetic.

6. 1's Complement

Definition

The 1's Complement of a binary number is obtained by inverting every bit.

Rule:

$0 \rightarrow 1$

$1 \rightarrow 0$

Example 1

Binary:

00000101

Invert all bits:

11111010

This is the **1's Complement**.

Example 2

Find the 1's Complement of:

10101010

Invert:

01010101

Problem with 1's Complement

Just like Sign-Magnitude, it also has **two representations of zero**.

Positive zero:

00000000

Negative zero:

11111111

Again, this is not desirable in computer systems.

7. 2's Complement

Definition

The 2's Complement of a binary number is obtained by taking the 1's Complement and then adding 1.

Formula:

2's Complement = 1's Complement + 1

Example 1

Find the 2's Complement of:

00000101

Step 1:

1's Complement:

11111010

Step 2:

Add 1:

11111010

+

00000001

11111011

Therefore:

-5 = 11111011

Example 2

Find the 2's Complement of:

00001010

1's Complement:

11110101

Add 1:

11110110

Result:

-10 = 11110110

8. Why is 2's Complement Better?

Advantages:

- Only **one representation of zero**
- Simpler arithmetic
- Faster hardware implementation
- No separate subtraction circuit is needed
- Used in almost all modern CPUs

This is why processors such as Intel, AMD, ARM, and MIPS use **2's Complement** for signed integers.

9. Range of Numbers

Unsigned

Formula:

0 to $(2^n - 1)$

Example (8 bits):

0 → 255

Signed (2's Complement)

Formula:

$$-2^{(n-1)} \text{ to } (2^{(n-1)} - 1)$$

Example (8 bits):

$$-128 \rightarrow +127$$

Comparison

Bits	Unsigned Range	Signed (2's Complement) Range
4	0 to 15	-8 to +7
8	0 to 255	-128 to +127
16	0 to 65,535	-32,768 to +32,767

10. How to Convert a Negative Decimal Number into 2's Complement

Example:

Represent **-13** using **8 bits**.

Step 1

Write **+13** in binary:

00001101

Step 2

Take the 1's Complement:

11110010

Step 3

Add 1:

11110011

Answer:

-13 = 11110011

11. How to Find the Decimal Value of a 2's Complement Number

Example:

11111011

Step 1

Take the 1's Complement:

00000100

Step 2

Add 1:

00000101

Decimal value:

5

Since the original MSB was **1**, the number is negative.

Answer:

-5

12. Real-Life Analogy

Imagine a building elevator.

- Floors above ground are **positive**.

- Basement floors are **negative**.

The elevator needs a method to distinguish between the two directions.

Similarly, the computer uses the **MSB (Sign Bit)** and **2's Complement** to distinguish positive and negative values.

13. Common Mistakes

Students often:

✗ Forget to add **1** after taking the 1's Complement.

✓ Always remember:

Invert Bits

↓

Add 1

✗ Confuse Sign-Magnitude with 2's Complement.

Remember:

- Sign-Magnitude: only change the sign bit.
 - 2's Complement: invert **all bits** and add **1**.
-

✗ Forget the range formulas.

Always memorize:

Unsigned:

$$0 \rightarrow 2^n - 1$$

Signed:

$$-2^{(n-1)} \rightarrow 2^{(n-1)} - 1$$



Lecture Summary

Today, we learned how computers represent positive and negative numbers using different signed number representation methods.

We studied Unsigned Numbers, Sign-Magnitude Representation, 1's Complement, and 2's Complement. We also learned why 2's Complement is the standard representation used in modern processors and practiced converting between decimal and binary using these methods.

This knowledge is essential for understanding processor arithmetic, Assembly Language programming, and how CPUs perform operations on signed integers.

In the next lecture, we'll learn **Binary Arithmetic using 2's Complement**, including addition and subtraction of signed numbers, overflow detection, and practical ALU examples.

Thank you for watching, and I'll see you in the next lecture.
